Spec to Inspector

The customer's own defect specification, enforced and cited on every reject.

REJECT **Clause 5.5 Damaged component or board** page 5 of the specification
Region 1: clause 5.5 rejects 'damage' in any zone, whatever its size.

A reject from the laptop prototype[1]: the clause, its page in the specification[2], and the rule's own sentence. Code decides; the language model only names the defect and the clause.

Theme and track	Workplace AI · Battlefield Thunderstorm (ASUS UGen Intel Arc Pro B70 32GB) · Global Online
Team	Brian Isaac Kam (solo)
Code	https://github.com/BrianIsaac/spec-to-inspector
Demo video	https://youtu.be/1RTXhtMX2sg

[1] Screen: docs/screens/page-01-reject-clause-line.png, the laptop prototype's console on VisA PCB4 test board 018, 11 October 2026. [2] The specification it cites is a demonstration document written for this entry (specs/pcb-spec.pdf in the repository).

AT A GLANCE

One specification in, one inspector out

Every decision is traceable to a clause of the customer's own document. Built from parts Intel's own stack lists for the Arc Pro B70[2], and measured end to end on the organiser's machine[1].

- **The problem.** A supplier's quality engineer turns each customer's defect specification into inspection rules by hand, and a reject that cannot name its clause is argued from memory.
- **What it does.** Parses the specification into typed clauses; an anomaly model finds the defect; code measures it; a vision-language model names the defect and the clause; a rule applies the clause's limit; the engineer's correction is kept against the clause.
- **Why the B70.** One card holds an 8B vision-language model at FP16 or FP8 with its context, as measured, and a second carries the anomaly model, on the supplier's own premises, where a customer's confidential specification stays.

189 of 200
defective boards rejected (178) or referred to an engineer (11)

34.1%
right clause cited: 46 of 135 single-defect boards (95% interval 26.6% to 42.4%)

16 of 201
good boards wrongly rejected (8.0%)

11.8 min
from a new specification to a working inspector, one part

Not held out: the thresholds were chosen on the same test split. The clause figure is low, misses its target and is printed as measured; page 17 has the table, and page 12 is Stage I's, as history.

JUDGING CRITERION (WEIGHT)[3]	PAGES
Innovation and Creativity (30%)	4, 5, 15
Commercial Potential and Feasibility (35%)	12, 13, 14, 16, 17, 18, 19
Practical Value (25%)	3, 6, 11, 18
Storytelling and Presentation (10%)	the whole deck; the video; page 6

WHAT THE CONTEST ASKS THE DECK TO HOLD	PAGES
Scenario and problem	3, 4
Tools and methods	5 to 11
Expected outcomes	12, 17
Hardware and software architecture	13, 14, 16
References and the GitHub link	20; the link also on page 1

Code and its documentation: https://github.com/BrianIsaac/spec-to-inspector

[1] Figures: results/stage2/final/bench.json, the final run of 7 November 2026 on the organiser's machine, under eval/PROTOCOL-stage2.md; the lenient and per-region figures on page 17. [2] Intel's list for the B70: llm-scaler's model table, reference 11. [3] Criteria and weights: the contest's page, reference 1, read 11 October 2026.

SCENARIO AND PROBLEM

The engineer's problem: which clause did this board fail?

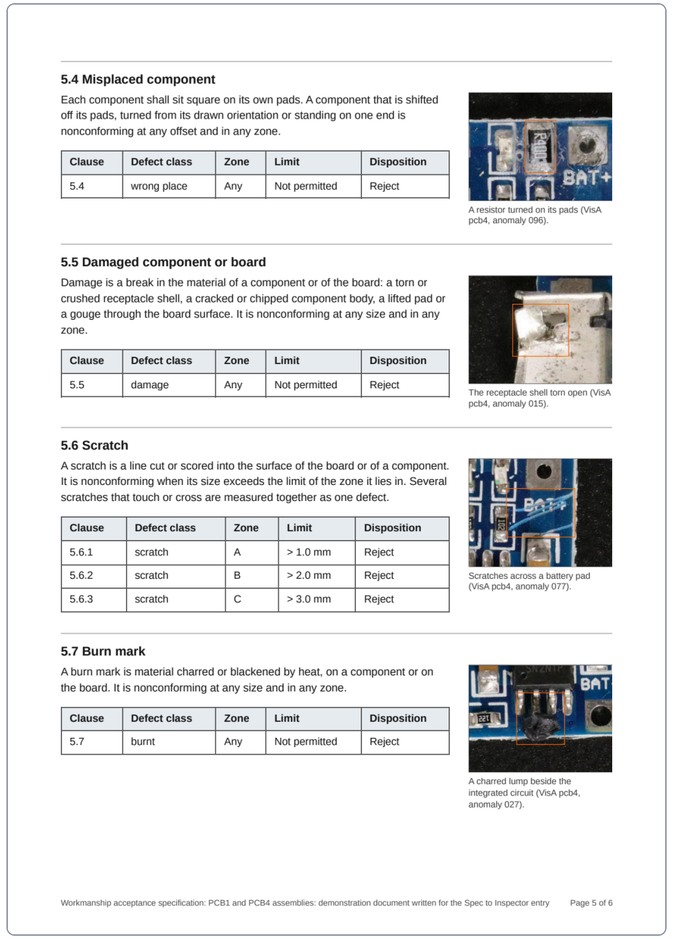
A supplier's quality engineer turns a customer's defect specification into inspection rules by hand, and loses every argument about a reject that cannot name its clause.

"The customer sends a 30-page defect specification with photos. It takes us weeks to turn it into inspection rules. When we reject a part, nobody can say which clause it failed, and when they reject ours, we argue from memory."

A quality engineer at a component supplier: a persona written for this proposal, not a quotation from a person.

What it costs, in words:[3]

- **Delay.** A new part cannot be inspected until someone has read the document and written its rules.
- **Drift.** The rules live in an inspection recipe, the limits in a document; after the next revision they no longer say the same thing.
- **Disputes.** A reject that names no clause is settled by whoever remembers the specification better.



5.4 Misplaced component

Each component shall sit square on its own pads. A component that is shifted off its pads, turned from its drawn orientation or standing on one end is nonconforming at any offset and in any zone.

Clause	Defect class	Zone	Limit	Disposition
5.4	wrong place	Any	Not permitted	Reject

A resistor turned on its pads (VisA pcb4, anomaly 096).

5.5 Damaged component or board

Damage is a break in the material of a component or of the board: a torn or crushed receptacle shell, a cracked or chipped component body, a lifted pad or a gouge through the board surface. It is nonconforming at any size and in any zone.

Clause	Defect class	Zone	Limit	Disposition
5.5	damage	Any	Not permitted	Reject

The receptacle shell torn open (VisA pcb4, anomaly 015).

5.6 Scratch

A scratch is a line cut or scored into the surface of the board or of a component. It is nonconforming when its size exceeds the limit of the zone it lies in. Several scratches that touch or cross are measured together as one defect.

Clause	Defect class	Zone	Limit	Disposition
5.6.1	scratch	A	> 1.0 mm	Reject
5.6.2	scratch	B	> 2.0 mm	Reject
5.6.3	scratch	C	> 3.0 mm	Reject

Scratches across a battery pad (VisA pcb4, anomaly 077).

5.7 Burn mark

A burn mark is material charred or blackened by heat, on a component or on the board. It is nonconforming at any size and in any zone.

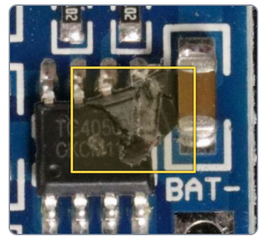
Clause	Defect class	Zone	Limit	Disposition
5.7	burnt	Any	Not permitted	Reject

A charred lump beside the integrated circuit (VisA pcb4, anomaly 027).

Workmanship acceptance specification: PCB1 and PCB4 assemblies: demonstration document written for the Spec to Inspector entry Page 5 of 6

?

which clause



[1] Left: page 5 of the demonstration specification, specs/pcb-spec.pdf, written for this entry in the style of a customer's document. [2] Right: VisA PCB4 test board 018 as the console shows it (docs/screens/page-05-2-finds.png); VisA is Amazon's dataset, CC BY 4.0, reference 6. [3] No market-size figure is printed: none was found at a source.

What exists, and the step nobody ships

Detection, explanation and criteria typed into a prompt all exist. In the sources we read to 11 October 2026[3] we found nothing that takes in the customer's own specification and quotes its clause and limit on every reject.

NEAREST NEIGHBOUR[1]	WHAT IT DOES, IN ITS OWN WORDS[2]	WHERE IT STOPS
Intel Open Edge Platform 2026.2, 10 September 2026	"defect detection into actionable quality insights through agentic workflows, VLM-based explainability"	Explains a defect; no customer document is read and no clause is cited.
Intel with Advantech, Pro Day fact sheet, March 2026	On Arc Pro B-Series: a VLM with retrieval "to help engineers analyze defects, reference documentation, and identify likely root causes"	Retrieves documentation for an engineer to read; binds no accept or reject to a clause and its limit.
PCBA Standard-to-Real Challenge (ASUS and NTU), ACM Multimedia 2026	"cross-domain PCBA inspection VQA … defect identification, quantitative reasoning, and defect cause/treatment"	Answers questions, scored as a benchmark. Ours is the step after: a customer's own contract specification, cited on each decision.
GLLS, arXiv 2607.03817, 4 July 2026	"structured specifications in the inference context"; decisions "traceable to explicit visual evidence"	Traceable to visual evidence, not to a clause of a customer's document.
TunnelMIND, arXiv 2604.27928, 30 April 2026	Measures a defect, grades severity "under expert knowledge constraints", writes a "retrieval-grounded explanation"	Tunnels; thresholds set by hand, not parsed from a customer's document; no memory of overrides.
Aqrose AIDI 3.4, 17 July 2026; VISION 2026 notice, 3 September 2026	"Just describe your needs in natural language"; an AI Review for "consistency, traceability and decision efficiency"	Needs are typed by a person; no document is read and no clause cited.
Overview AI OV Spark, 6 October 2026	"Describe your part and what should pass or fail."	A setup assistant on the camera; its record is a capture library and an audit log, not a clause.
ZEDEDA, 19 March 2026 (older: context)	A fixed 8B VLM on Jetson Thor; the inspection policy is "a natural language prompt"	The rules are a prompt an engineer writes.

Not claimed as new, because each exists: anomaly detection from good images (Anomalib); a VLM explaining a defect (Intel); corrections fed back; answers that cite a source document; and "measure, a rule decides, a model explains" (TunnelMIND does it for tunnels). The incumbent vision suites (MVTec, Keyence, Cognex, Landing AI): nothing found that reads a specification or cites a clause.[2]

Claimed: the narrow chain. The customer's PDF becomes typed clauses; code measures; the model names the clause; code applies its limit; the reject quotes clause and limit; the override is kept per clause.

	No document: rules typed or trained	Public standards or manuals, retrieved	The customer's own specification, parsed
Cites clause and limit on each decision	none found	none found	**Spec to Inspector**
Does not	Intel OEP 2026.2 · Aqrose · Overview · ZEDEDA · incumbents	Intel with Advantech · the PCBA challenge · GLLS · TunnelMIND	none found

[1] Each row's source, with its URL, is on page 20: Intel references 14 and 15; the challenge 16; GLLS 17; TunnelMIND 19; Aqrose 20; Overview 21; ZEDEDA 22. [2] Quotations were read at those pages on 11 October 2026; the incumbents' 2026 releases were searched on 9 October 2026 and not every page could be opened. [3] Simplified-Chinese platform vendors were not searched.

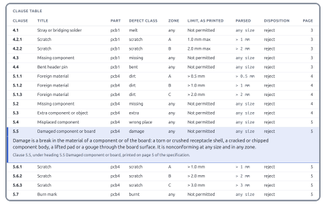
The specification is the source of truth

The anomaly model finds, code measures, the vision-language model names the defect and the clause, a rule decides, and the engineer corrects. No number comes from the language model.

1 · PARSE

The specification becomes clauses

The customer's PDF[2] is parsed into a typed table: clause, defect class, zone, limit, page.


CLAUSE TABLE

CLAUSE	TITLE	PART	DEFECT CLASS	ZONE	LIMIT, AS PRINTED	PARSED	DISPOSITION	PAGE
4.1	Stray or bridging solder	pcb1	melt	any	Not permitted	any size	reject	3
4.2.1	Scratch	pcb1	scratch	A	1.0 mm max	≤ 1 mm	reject	3
4.2.2	Scratch	pcb1	scratch	B	2.0 mm max	≤ 2 mm	reject	3
4.3	Missing component	pcb1	missing	any	Not permitted	any size	reject	3
4.4	Bent header pin	pcb1	bent	any	Not permitted	any size	reject	3
5.1.1	Foreign material	pcb4	dirt	A	> 8.5 mm	> 8.5 mm	reject	4
5.1.2	Foreign material	pcb4	dirt	B	> 12 mm	> 5 mm	reject	4
5.1.3	Foreign material	pcb4	dirt	C	> 2.0 mm	> 2 mm	reject	4
5.2	Missing component	pcb4	missing	any	Not permitted	any size	reject	4
5.3	Extra component or object	pcb4	extra	any	Not permitted	any size	reject	4
5.4	Misplaced component	pcb4	wrong place	any	Not permitted	any size	reject	5
5.5	Damaged component or board	pcb4	damage	any	Not permitted	any size	reject	5
5.6.1	Scratch	pcb4	scratch	A	> 1.0 mm	> 1 mm	reject	5
5.6.2	Scratch	pcb4	scratch	B	> 2.0 mm	> 2 mm	reject	5
5.6.3	Scratch	pcb4	scratch	C	> 3.0 mm	> 3 mm	reject	5
5.7	Burn mark	pcb4	burnt	any	Not permitted	any size	reject	5

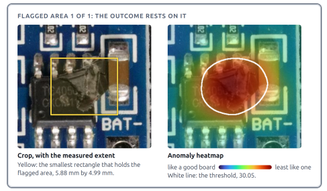
Damage is a break in the material of a component or of the board: a torn or crushed receptacle shell, a cracked or chipped component body, a lifted pad or a gouge through the board surface. It is nonconforming at any size and in any zone.

Clause 5.5, under heading 5.5 Damaged component or board, printed on page 5 of the specification.

2 · FIND

The anomaly model finds

Trained on good boards only, it flags what differs from good and where.



3 · MEASURE

Code measures

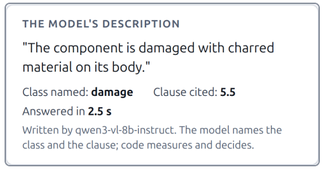
The flagged area's extent in millimetres and its zone, from the specification's own calibration.



4 · NAME

The model names

A vision-language model reads the crop and the clause table and names a defect class and a clause.



5 · DECIDE

A rule decides and cites

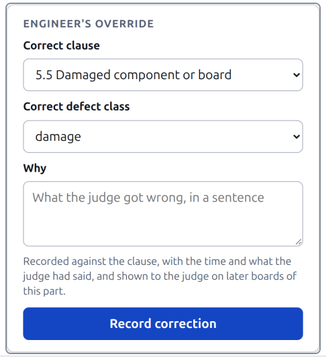
Code applies the clause's limit to the measurement and quotes the clause on the reject.



6 · CORRECT

The engineer corrects

An override is stored against the clause with its reason, and shown to the judge on later boards.

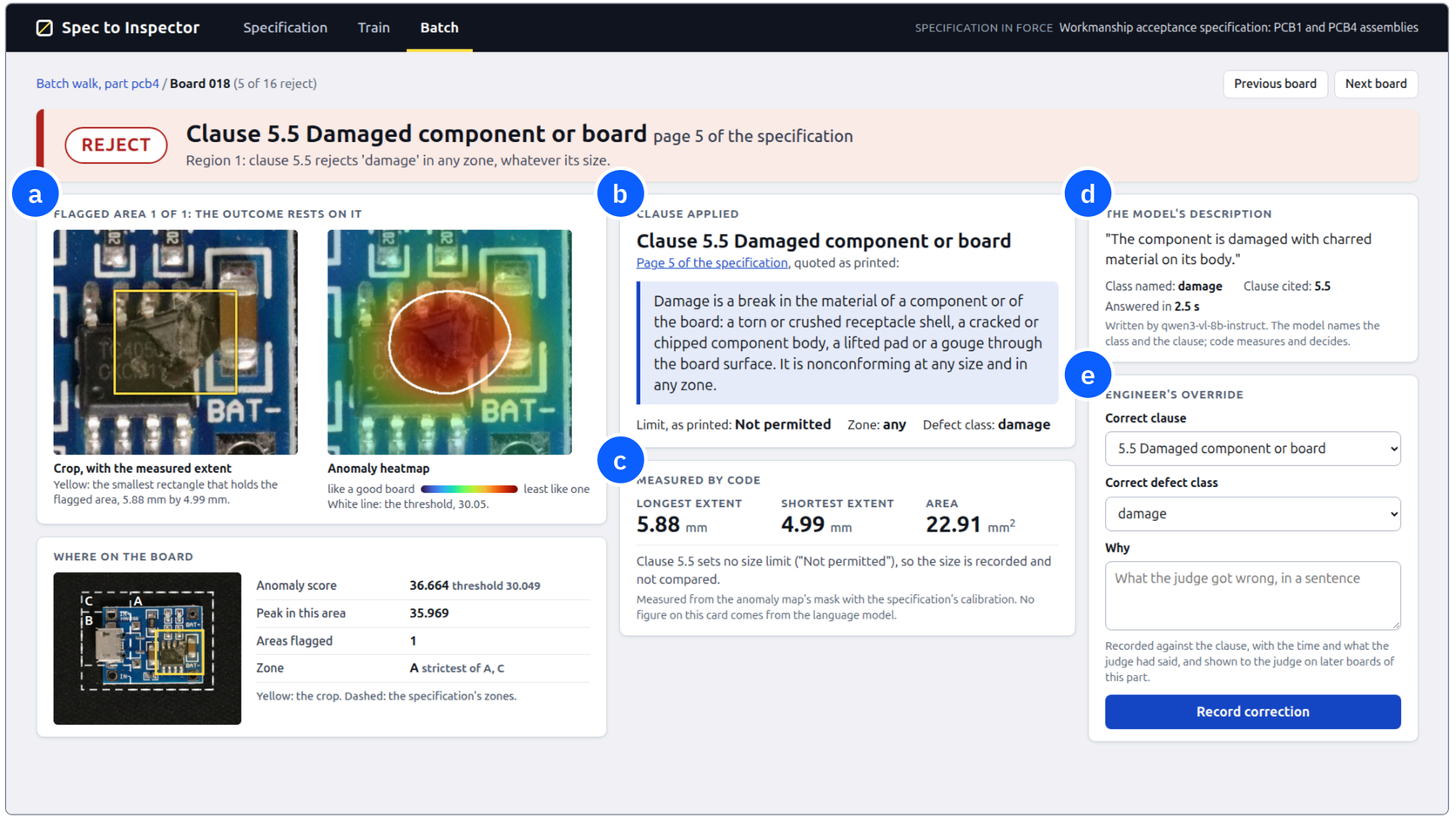


The customer's own specification, cited on every reject.

[1] Screens: docs/screens/page-05-1-specification.png to page-05-6-corrects.png, the console's own panel for each step, on VisA PCB4 test board 018 (11 October 2026). [2] The specification is the demonstration document, specs/pcb-spec.pdf in the repository.

One reject, end to end

A reject[2] that shows its grounds: the crop, the clause as the customer printed it, the size measured by code, and a way to correct it.



a **The crop and the heatmap.** Where the anomaly model flagged, with the measured rectangle drawn on the photograph.

b **The clause, quoted as printed**, with its page in the specification, its limit, zone and defect class.

c **Measured by code** from the anomaly map's mask. This clause sets no size limit, so the size is recorded and not compared.

d **The model's one line.**[4] It names the class and the clause[3] and writes no figure.

e **The override.** The engineer picks the right clause and says why.

[1] Screen: docs/screens/page-06-reject-card.png, the console on VisA PCB4 test board 018, 11 October 2026. [2] The board was chosen by rule, not by eye: the first reject of a 24-board batch on which the judge named VisA's own defect type. [3] How often it does is on page 12. [4] The answer time on the card is that run's, not the bench's.

Method 1: the specification becomes clauses

A born-digital specification parses to a typed clause table without anyone retyping it: 6 pages, 16 clauses under 11 numbered headings and 5 zones[2], in 7.9 s on the CPU[3].

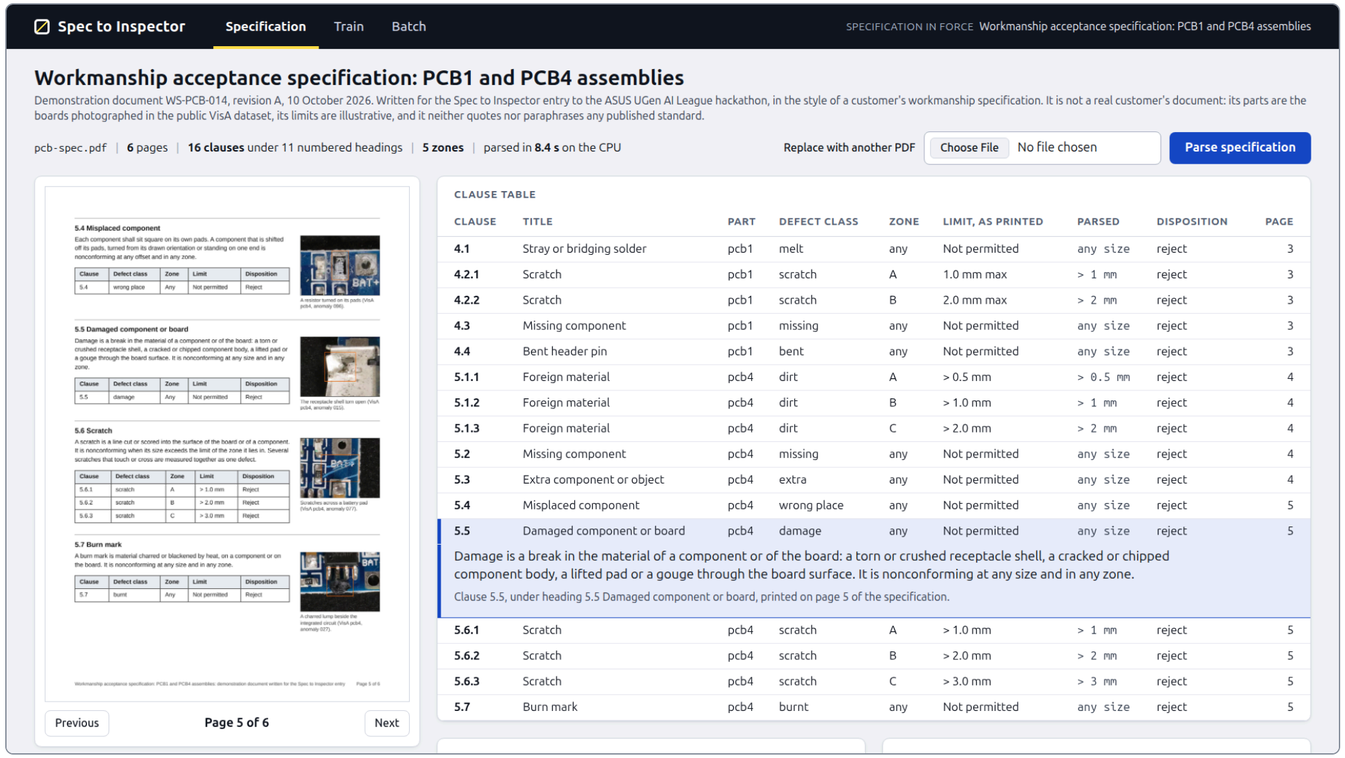
The clause, as a record: id and heading; title; part; defect class; zone; the limit as printed and as a number with its unit and comparison; disposition; page; the source text.

The parser: docling, with heading levels kept so that 5.1.2 stays under 5.1. Tables of limits are read as numbers, not prose: "1.0 mm max" and "> 1.0 mm" both become `> 1 mm`.

What is checked: every clause recovered and every limit parsed with its unit. The bench refuses to run unless today's parse equals the tracked clause table.

The calibration and the zones are declared in the specification too, so a size in millimetres and a zone are the customer's, not ours.

Scanned specifications[4] go through an OCR vision-language model on the card: not yet run.



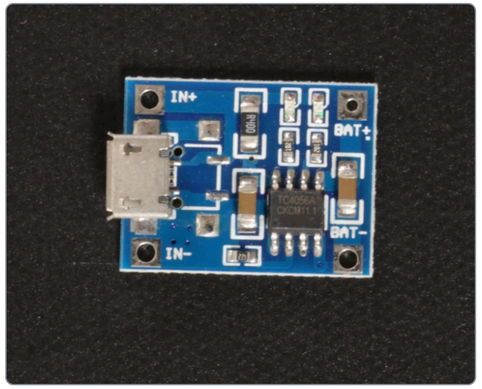
The PDF's page on the left, the parsed table on the right; clause 5.5 is open, quoted with its page.[1]

[1] Screen: docs/screens/page-07-specification-clause.png. [2] Counts: results/stage1/spec.json (10 October 2026). [3] Parse time: results/stage1/bench.json (11 October 2026); the run on screen took 8.4 s. docling 2.135.0, MIT: reference 9. [4] The scanned-page path is planned for Stage II and is not in the Stage I prototype.

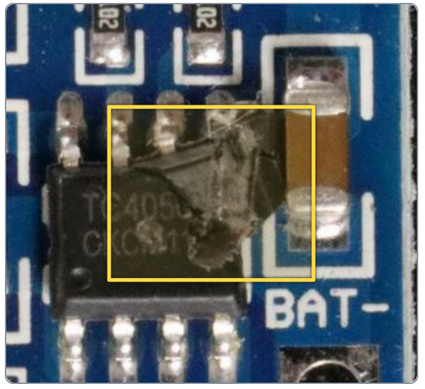
Method 2: the anomaly model learns "good" from good parts

The inspector needs only good images to start; a defect is what differs from good.

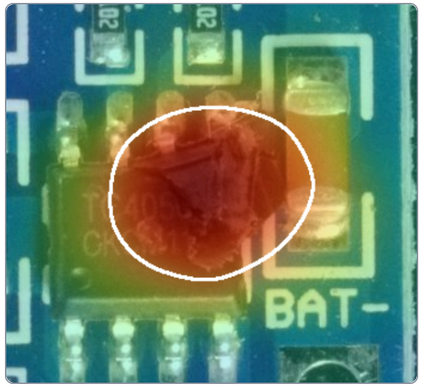
- **Anomalib 2.7.0** (Intel, Apache-2.0), PatchCore with its default `wide_resnet50_2` backbone, trained on the 904 good images of a VisA PCB part at 256 px. One fit takes 12.2 minutes on the laptop.[2]
- **Why the anomaly model finds and the language model does not.** The VAND 4.0 challenge report[3] (3 October 2026): "the best off-the-shelf VLM approach trails specialized models by ~28 AP, confirming that currently VLMs cannot replace fine-tuned detectors".
- **What it gives the next step:** a score for the board, a map of where it differs, and the flagged area's outline.
- **A new part with only 10 to 50 good images** is the few-shot case: not measured.



A good board: all the model is trained on[1]



A test board: the flagged area



Its anomaly map; white: the threshold

IMAGE AUROC, ONE RUN PER PART[2]	PCB1	PCB2	PCB3	PCB4
This prototype: PatchCore, `wide_resnet50_2`, 256 px	93.0	92.3	92.1	98.7
The Anomalib team's published ResNet18 figure, 256 px	94.9	92.4	99.0	99.0
Difference, in points	-1.9	-0.1	-6.9	-0.3

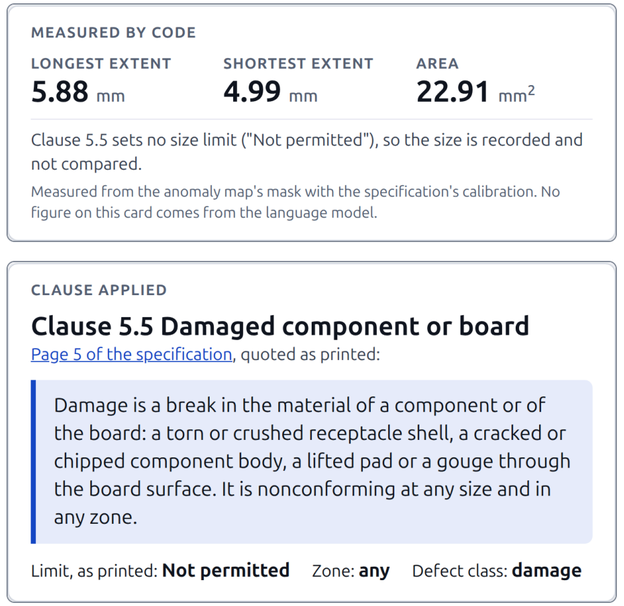
Not like for like: the published figure is the mean of three runs with another backbone. PCB3 is the cell we did not come near: every configuration we tried stayed more than five points under it. The judge and the clause figures use PCB1 and PCB4.

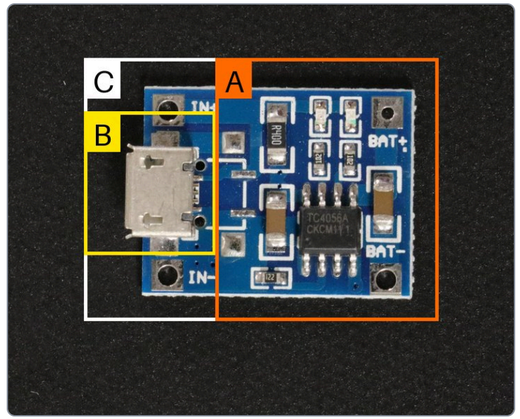
[1] Images: VisA PCB4 (Amazon, CC BY 4.0) as the console shows them: good board 0107 from docs/screens/extra-accepted-card.png, test board 018 from page-05-2-finds.png. [2] Training figures and AUROC: results/stage1/anomaly.json (10 October 2026); the published figure is Rolih et al., reference 5, its column 'Single model 256'. [3] The VAND 4.0 report: reference 18, read 11 October 2026.

Method 3: code measures; the model does not guess numbers

Every size on a reject is measured from the anomaly map's mask in calibrated millimetres, and accept or reject is the clause's limit against that measurement, decided by code.

- **Calibration** is declared per part in the specification: PCB4's image covers 46.7 mm across[3]. The scale is the customer's statement, printed on the card.
- **Size** is the long side of the smallest rectangle, at any angle, that holds the flagged area: the specification's own definition.
- **Zone** comes from the specification's zone map; where zones overlap, the most critical applies.
- **The limit check is a rule.** A size over the clause's limit rejects; "not permitted" rejects at any size. An area outside every zone is referred to an engineer, and so is an area with no outline when its clause sets a size limit: neither is guessed.
- **What it can resolve.** The anomaly map is 256 px across, 0.18 mm a pixel on PCB4[4], and it lights the neighbourhood of a defect, so a measured extent is often larger than the defect a person sees. A size near a limit is a case for the engineer; the models trained on the machine keep the 256 px map.





The specification's zones for PCB4[2]: A, components; B, the receptacle; C, pads and margin

[1] Screens: docs/screens/page-05-3-measures.png and page-05-5-cites.png (board 018). [2] Zones: specs/figures/pcb4-zones.jpg, from the demonstration specification. [3] Calibration: specs/pcb-spec.clauses.json; map size: results/stage1/anomaly.json. [4] The map's pixel is their quotient, not a measured resolution.

Method 4: the judge names the defect and the clause

A vision-language model[4] reads the crop with the whole clause table in context and returns a structured answer; a rule then decides from the clause's limit and the measured size.

What the model is shown, in this order[1]

1. The instructions and the whole clause table, each clause with its source text: 2,260 tokens, the same for every board, so a server can cache it.
2. The part and its numbered headings.
3. Up to three engineer's corrections recalled for this part.
4. The measured facts, as JSON: the code's numbers, not the model's.
5. The crop, then the same crop under the anomaly heatmap.

What keeps it honest

- The answer is constrained to a JSON schema; its one line may hold no digit.
- A clause that is not in the table, or a class that is not that clause's, makes the board "needs review".
- The model may say that no clause applies; that too is "needs review". It can never accept a board.

Its answer for board 018[2]

```
{
  "reason": "The component is damaged with charred material on
its body.",
  "defect_class": "damage",
  "clause_id": "5.5"
}
```

The rule's sentence, written by code

```
clause 5.5 rejects 'damage' in any zone, whatever its size
```

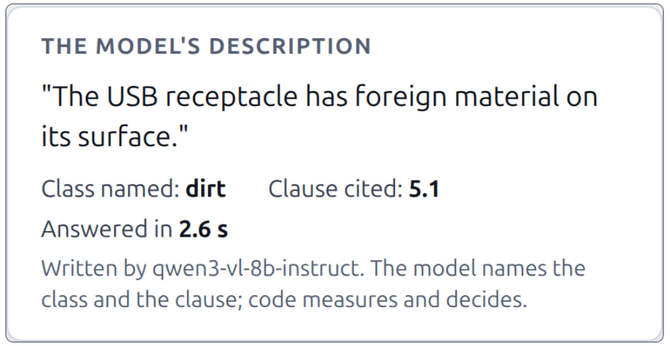
Measured on the laptop:[3] a prompt of 4,683 tokens at the median, an answer of 39 tokens in 3.33 s asked cold (2.55 s with the clause table cached). The count of bench citations whose class was not the cited clause's own: 0. As a contract it holds; as a classifier at 4-bit it is weak, and page 12 says how weak.

[1] Prompt order: src/spec_to_inspector/judge/prompt.py. [2] The answer shown is board 018's (docs/screens/page-05-4-names.png). [3] Token counts and times: results/stage1/judge.json (11 October 2026) and bench.json (11 October 2026); laptop, llama.cpp, Qwen3VL-8B-Instruct-Q4_K_M.gguf. [4] Qwen3-VL-8B-Instruct, Apache-2.0: reference 7.

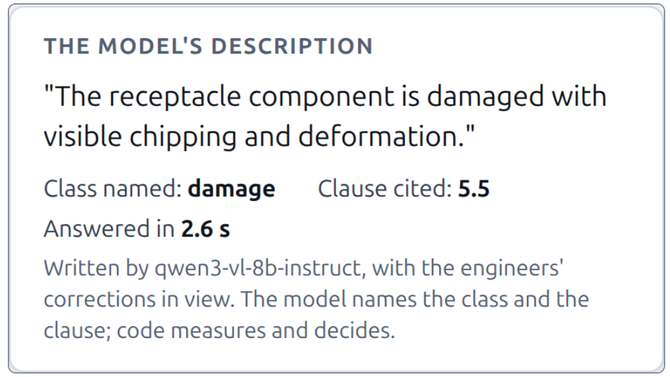
Method 5: the engineer's correction is kept

The engineer's correction is stored, shown to the judge, and a similar defect on another board then cites the corrected clause. Every override is logged against its clause: the audit trail a customer asks for.

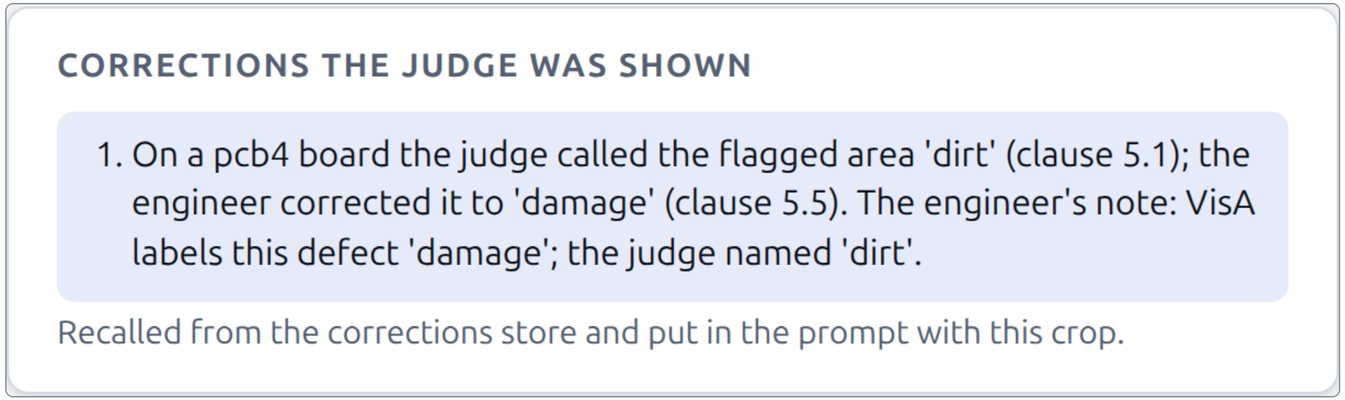
- **Stored:** the board and area, what the judge had said, the corrected class and clause, the engineer's note, the time. The judge's own decision is kept beside it, not overwritten.
- **Recalled:** the part's most recent corrections, three at most, in the judge's prompt.
- **Measured:**[3] in 7 of 7 scripted pairs the second board, judged wrong before, cites the corrected clause after one override.
- **And its limit, measured too:** the same single correction also moved 14 of 19 control boards that truly are the class corrected away from; all 10 of 10 controls of the class corrected to held. With this model a correction acts as a rule about the class, not as an example.
- **So we do not claim** that corrections raise accuracy or teach the judge to tell two defects apart. No retraining happens in Stage I.



Board 012 before the correction of board 005[1]



Board 012 judged again afterwards



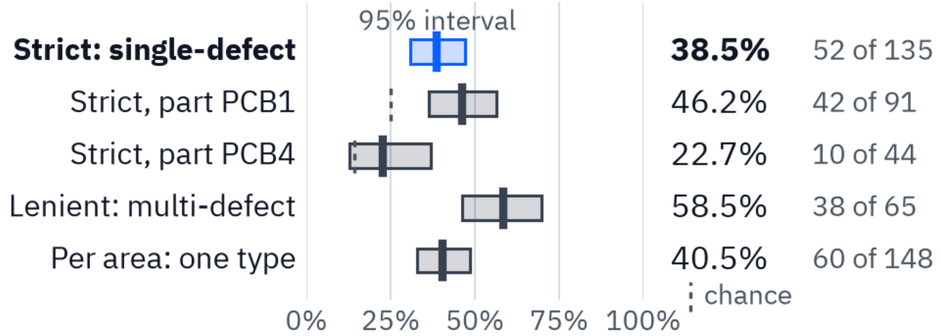
The correction, as the judge was shown it and as the card prints it[2]

[1] Screens: docs/screens/page-11-3-second-board-before-model.png, page-11-4-second-board-after-model.png and page-11-4-second-board-after-shown.png: boards 005 and 012. [2] The note on screen was written by the walk's script from VisA's label, not by an engineer. [3] Counts: results/stage1/corrections.json (11 October 2026), each pair asked cold.

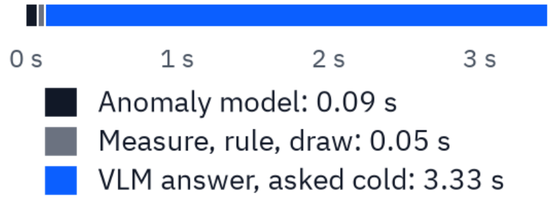
Stage I: what the laptop prototype measured

The pipeline runs end to end today on a 12 GB laptop GPU[2]. These are its figures, the low one included.

FIGURE	MEASURED ON 200 DEFECTIVE AND 201 GOOD VISA TEST BOARDS, PARTS PCB1 AND PCB4[1]
Right clause cited, strict	**52 of 135 single-defect boards, 38.5%** (95% interval[3] 30.7% to 46.9%). PCB1 42 of 91; PCB4 10 of 44
Defective boards stopped	188 of 200: 174 rejected, 14 referred to an engineer. The 12 that passed were not flagged by the anomaly model
Good boards wrongly rejected	16 of 201, 8.0% (5.0% to 12.5%); 9 more referred
A flagged board, end to end	3.49 s median, 4.03 s p95 (216 boards). A board that is not flagged: 86 ms
The VLM's answer, asked cold	3.33 s median, 3.84 s p95, at a prompt of 4,683 tokens (218 answers)
Anomaly model alone	20.3 to 20.6 images a second, from file to score
Peak GPU memory	judge 8,780 MiB; anomaly model while scoring 2,028 MiB; training 8,003 MiB
Specification to working inspector	12.3 min for one part: 7.9 s to parse, the rest training
Run twice	the same answer on 401 of 401 boards



Right clause cited, with 95% intervals. The lenient figure counts a clause of any of a board's defect types and is not a second headline.



Where a flagged board's time goes, at each stage's median.

How to read it. The clause is scored on the most anomalous area, with an empty corrections store; a judge citing at random would be right on 25.0% of PCB1's boards and 14.3% of PCB4's. The anomaly model is not what holds the figure down: on 109 of the 135 boards its area lies on the labelled defect, and the judge cites the right clause on 48 of those (44.0%).

Development figures. The thresholds were chosen on the same test split and the prompt was written with sight of some of its images. Labels are VisA's own defect annotation mapped to clauses by a written table, 20 spot-checked by hand (20 of 20 agree). The laptop cannot show the VLM above 4-bit or beyond a 16,384-token context: Stage II measures both.

[1] Every figure: results/stage1/bench.json, written by `uv run spec-to-inspector bench` on 11 October 2026 under eval/PROTOCOL.md (written before the run; SHA-256 ebbd8cc1cb25...); images a second: results/stage1/anomaly.json. [2] Laptop: RTX 5070 Ti Laptop GPU, 12 GB; llama.cpp serving Qwen3VL-8B-Instruct-Q4_K_M.gguf with a 16,384-token context. [3] Intervals are 95% Wilson intervals.

One codebase, two backends

CUDA on the laptop for Stage I; the Arc Pro B70 through Intel's own stack for Stage II. Which backend runs is a setting, not a code change.

	Stage I: the laptop MEASURED	Stage II: two Arc Pro B70s, one bare-metal host MEASURED
CONSOLE	FastAPI and Jinja2, rendered on the server, nothing to compile · the corrections store is one SQLite file	
PIPELINE	parse → find → measure → name → decide → correct: one Python package, one command line, `spec-to-inspector`	
JUDGE CLIENT	one OpenAI-compatible client with a JSON-schema answer; the server's address and model are settings	
VLM SERVER	llama.cpp `llama-server` · Qwen3-VL-8B-Instruct at 4-bit (Qwen3VL-8B-Instruct-Q4_K_M.gguf) · 16,384-token context, one slot	Intel llm-scaler's vLLM image at the pinned tag 0.26.0-b1, on card 1 · Qwen3-VL-8B-Instruct at FP16, FP8 as a declared comparison · 32,768-token context
ANOMALY MODEL	Anomalib 2.7.0 PatchCore on PyTorch 2.14.1+cu130, in FP32[1]	the same Anomalib on PyTorch 2.14.1 XPU in FP32, on card 0; OpenVINO was measured, and nothing in the pipeline runs through it
PARSER	docling on the CPU	docling on the CPU · scanned pages through PaddleOCR-VL on the card: planned, not run
HARDWARE	NVIDIA GeForce RTX 5070 Ti Laptop GPU, 12 GB	Intel Arc Pro B70, two whole cards of 32,656 MiB[3]

- **Why Qwen3-VL-8B-Instruct.** Intel's llm-scaler[1], "running on Intel Arc Pro B60 and B70 GPUs", lists `Qwen/Qwen3-VL-8B-Instruct` at FP16, FP8 and Int4; it is Apache-2.0; and llama.cpp serves the same model on the laptop, so both stages judge with one model.
- **Why upstream PyTorch XPU and llm-scaler's vLLM.** IPEX-LLM and Intel Extension for PyTorch are archived[2]; their last commits are 28 January and 30 March 2026. OpenVINO gives the same scores to within 0.0004 at about half PyTorch XPU's speed, so PyTorch XPU is the path on this machine.
- **What is now shown.** We found no published figure for Anomalib or for any vision-language model on a B70; pages 14 to 17 give what the machine measured for both. Not yet run on the machine: a scanned specification, a good board's picture beside the crop, and a store of corrections.

[1] Stage I versions: results/stage1/anomaly.json and bench.json. llm-scaler's supported-model table and its 'running on Intel Arc Pro B60 and B70 GPUs': reference 11, read 11 October 2026. [2] Archived repositories: reference 24, read 11 October 2026. [3] The Stage II column: results/stage2/week1/machine.md, results/stage2/week2/README.md and results/stage2/final/SETTING.md.

A card each: the memory as measured

The pipeline ran on two whole cards, the judge at FP16 on one and the anomaly model on the other. The plan's 80% share was refused for a 32K context, so the judge's server takes 90% of its card.

What the environment was. Two whole Intel Arc Pro B70s in one bare-metal host, reached over ssh[1]: physical functions, 4 virtual functions possible and 0 made, 32,656 MiB on each; Intel Xeon 6787P, 344 threads, 251 GiB of memory; Ubuntu 26.04.1 LTS. The judge's container is given only card 1, so the two never share a card. The plan had assumed one whole card in a virtual machine, reached through a booked session, with training before the judge was loaded.



Card 1.[2] At FP16 vLLM loads 17.34 GiB of weights and leaves 6.16 GiB of cache: 44,800 tokens, 1.37 times a 32K context, where the plan[3] had about 55,000. At the plan's 80% it leaves 2.97 GiB and refuses 32K. The plan did not count what vLLM sets aside for peak activation and non-torch memory, 5.2 GiB; how much of it is the vision encoder was not measured. FP8, the declared comparison: 10.88 GiB of weights, a cache of 91,712 tokens, 2.80 contexts against the plan's 3.

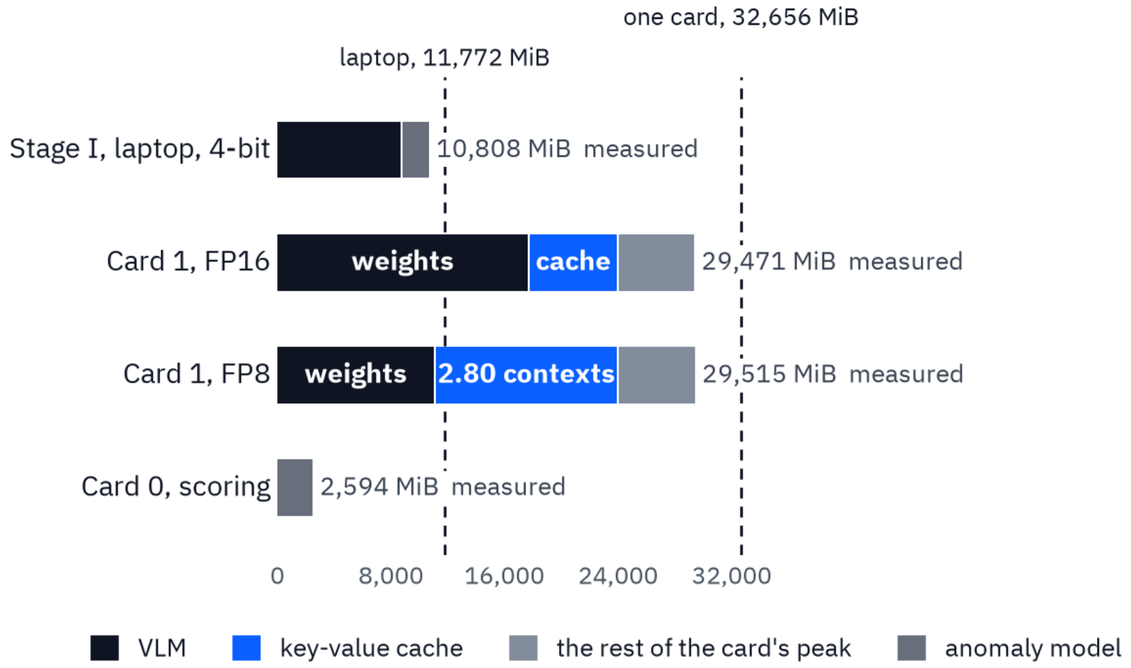
Card 0. Training peaks at 10,345 to 10,785 MiB over 8 trainings; the plan carried the laptop's figure over, so it has no bar. The model scores 29.61 to 30.04 images a second at batch 1 and 34.69 to 35.22 at batch 16, the card at 2,594 MiB in the final run and 2,899 to 4,069 MiB in week 2's own. **Across a whole batch:** memory per stage is to be measured before 2 December 2026 at 11:30 PLAN

[1] The machine: results/stage2/week1/machine.md, read 6 November 2026. [2] Card peaks, by `xpu-smi`: results/stage2/final/bench.json, results/stage2/comparison-fp8/bench.json, results/stage2/week2/run-1/card-pytorch.json. vLLM's lines: the logs under results/stage2/week1/raw/ and comparison-fp8/raw/. [3] Planned: deck/stage2-plan.json (reference 7), in MiB.

Why this needs the B70

One card holds an 8B vision-language model at FP16 or FP8 with a 32K context, and a second holds the anomaly model, on the supplier's premises, where a customer's confidential specification stays.

- **Precision.** The laptop's 12 GB holds the judge at 4-bit beside the anomaly model with little to spare (the card peaked at 11,395 MiB[1]). The weights are 17.34 GiB at FP16[2], which no 12 GB card holds, and 10.88 GiB at FP8, which leaves no room there for a context and the anomaly model. What the machine measured, in the protocol's words: strict clause-citation accuracy 34.1%, 46 of 135, 95% interval 26.6% to 42.4%, at FP16; FP8, a declared comparison: no figure separates them.
- **Context, more than once.** One question is 4,682 tokens, for a 6-page specification of 2,868 tokens; a 30-page one at that density is about 14,340. Every question in flight needs its own cache: the machine's holds 1.37 contexts of 32K at FP16 and 2.80 at FP8. The bench asks one at a time, so more than one in flight is to be measured PLAN
- **A card each.** The judge's server stays up on card 1 while the anomaly model trains on card 0: card 1 read 29,455.73 to 29,455.77 MiB over 1,830 readings.
- **On premises.** A customer's specification is a confidential contract document. A card in the supplier's own workstation keeps it there; a cloud model does not.
- **The evidence that exists, and its limit.** Backend.AI[3] measured 49.6 tokens a second on one B70 for "Qwen3 8B, 32K context, 4 concurrent": a text model, not a vision-language model, and the only published figure we found. The machine's own answer to one area, asked cold at FP16: median 2.12 s, 95th percentile 2.54 s. Throughput was not read: vLLM gives no generation rate. An integrated GPU may hold an 8B model in shared memory; we assert no figure for it.



Memory only: what each card holds, not how well it judges. Top bar: Stage I, on the laptop[1]. Lower bars: the machine's cards[2]; weights and cache are vLLM's lines, the total the card's peak.

[1] Stage I's figures: results/stage1/bench.json, spec.json and anomaly.json. [2] The machine: results/stage2/final/bench.json and FINAL.md, results/stage2/comparison-fp8/COMPARISON.md; vLLM's lines as on page 14. [3] Backend.AI, 'Intel Arc meets Backend.AI', June 2026, table row 'Qwen3 8B, 32K context, 4 concurrent': reference 13, read 11 October 2026.

Stage II: what ran on the organiser's machine

Weeks 1 to 3 ran ahead of their weeks[1], the final run under a protocol in force before it; week 4 runs before 2 December 2026 at 11:30.

WEEK 1 · 6 TO 12 NOVEMBER
DONE 6 NOVEMBER

Bring-up

Read the cards, installed the pinned environment, checked PyTorch XPU and OpenVINO, started the judge's server.

Output: the machine's shape, two whole cards of 32,656 MiB; the answer schema accepted with one fix.

WEEK 2 · 13 TO 19 NOVEMBER
DONE 6 NOVEMBER UTC

The anomaly model

PatchCore on PyTorch XPU in FP32 and through OpenVINO, four parts, two runs.

Output: 1.3 to 1.4 times the laptop's speed at the same scores: within 0.00053 on one checkpoint; training moves 14 of 802 boards by more than 0.001.

WEEK 3 · 20 TO 26 NOVEMBER
DONE 7 NOVEMBER

The judge at full precision

FP16 as the final setting and FP8 as a declared comparison, on the 200 labelled boards.

Output: 34.1% strict, 46 of 135; an answer in 2.12 s, median. FP8: no figure separates them. Throughput is not read.

WEEK 4 · 27 NOVEMBER TO
4 DECEMBER 2026

The full run PLAN

A whole batch end to end; memory per stage; the corrections loop across the batch.

Output, to be measured before 2 December 2026 at 11:30: the final video recorded from a session on the machine; the repository with its measured figures.

The protocol:[2] one setting, declared before the run and reported whatever it read. FP16, committed before the first run, and not the best of several.

Three runs, the third after a server restart: 343 counted fields, 0 differ.

The checkpoints held by digest (b71ff0c3..., cc571d9a...): any other voids a run.

The region columns regenerated from the machine's checkpoints: 1 row of 200 moved, PCB1 `bad-029`, 1.9e-6 above its threshold and one of the 46; without it, 45 of 135.

Three comparisons declared in advance, FP8, the image minimum, the designators pattern: each "no figure separates them".

Of the designators: the pattern with the validation shows the claim on these boards, the pattern alone does not.

Two dated additions, each before the run: the region columns with the threshold rule, and that pattern.

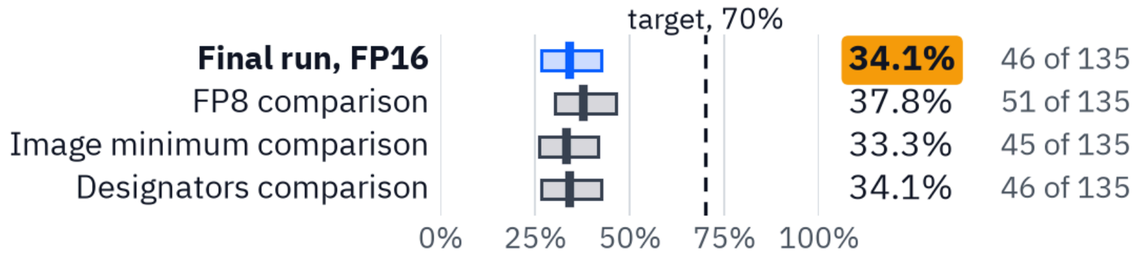
THE LAPTOP'S LIMIT	IN STAGE I[3]	WHAT STAGE II PUT IN ITS PLACE
The VLM's precision	4-bit	FP16, the final setting: 34.1% strict. FP8 as a declared comparison: no figure separates them
Its context	16,384 tokens, one question at a time	32,768 tokens, in a cache of 44,800 at FP16; more than one in flight: to be measured PLAN
The thresholds	chosen on the test split	still chosen on the test split, the only labelled data there is, so nothing here is held out
The run	401 boards of two parts	the same 401 boards, three runs; a whole batch and the corrections loop: to be measured PLAN

[1] Weeks and outputs: docs/stage2.md's table and each week's record. [2] The protocol: eval/PROTOCOL-stage2.md (SHA-256 ebbd8cc1cb25...) and its two additions; results/stage2/final/SETTING.md, results/stage2/checkpoints.json, results/stage2/region-columns/README.md, each COMPARISON.md. [3] Stage I's settings: results/stage1/bench.json.

Expected outcomes: the targets, and what the machine measured

The clause target was not met: 70% or more was set, the final run measured 34.1%, and no declared comparison separates from it.

MEASURE	TARGET, STAGE I[1]	MEASURED ON THE MACHINE[2]	READ
Right clause cited, strict	**70% or more**	**34.1%, 46 of 135**, 95% interval 26.6% to 42.4%	**Not met**
Beside it, under their own names	none set	The lenient figure, multi-defect: 70.8%, 46 of 65. The per-region figure: 36.2%, 54 of 149	Never pooled, no substitute
Defective boards stopped	no fewer, on a held-out split	189 of 200: 178 rejected, 11 referred; 11 passed	Not held out
Good boards wrongly rejected	4% or less, on a held-out split	8.0%, 16 of 201, 5.0% to 12.5%	**Not met**; not held out
Referred to an engineer	reported as it falls	11 of 200 defective, 10 of 201 good	Reported
Time for a flagged board	none set	2.33 s median at FP16; 2.05 s in the FP8 comparison	A reading, not a move
Specification to inspector	15 minutes or less	11.8 minutes, one part; the fit as recorded at training	**Met**
Peak GPU memory	within page 14's plan	Judge 29,471 MiB of 32,656, 29,784 after the restart; anomaly model 2,594 scoring, 10,785 training	Within each card, at 90%, not the plan's 80%
Size of the run	a 500-image batch, a 200-defect set	The 200 defective boards and 201 good, 401 in all, three runs. A whole batch: to be measured before 2 December 2026 at 11:30 PLAN	The set has run; the batch has not



Strict, with 95% intervals; each comparison tested on its own, uncorrected for there being three. Stage I's laptop figures stand on page 12 as history and are not set against these: the host, the precision, the server and the answer pattern all differ, and the experiment that would separate them is not run.

What the protocol's evidence says.[3] The anomaly model is not what holds the figure down: the most anomalous area lies on the labelled defect on 110 of the 135, and there the judge cites the right clause on 42 (38.2%, conditional). The judge names the wrong defect class. PCB1 reads 41 of 91, PCB4 5 of 44. FP8, the image minimum and the designators pattern: no figure separates them ($p = 0.125, 1, 1$).

What it does not say. Why the judge names the wrong class. A good board's picture beside the crop and the corrections loop have not run. The figure is not held out and not run blind: the protocol was written after a bring-up bench on these boards. It is one specification, one model, one precision, one machine.

[1] Targets: as Stage I set them, ours, kept in deck/quoted.json. [2] Measured: results/stage2/final/FINAL.md and bench.json, the final run of 7 November 2026; week 2's peak, results/stage2/week2/README.md. [3] Evidence: each comparison's COMPARISON.md and paired.json under results/stage2/; eval/PROTOCOL-stage2.md, 'What the figures are not'; docs/stage2.md.

Who pays, and what it replaces

A supplier's quality department pays, per line or per specification onboarded[4], for a box on its own premises.

- **The buyer:** the quality department of a component or board supplier that inspects to its customers' specifications.
- **What it replaces:** weeks of rule-writing by hand or by a vision integrator for each new specification, and disputes argued from memory.
- **The shape:** an on-premises workstation with one ASUS UGen Intel Arc Pro B70, because specifications are confidential. Every licence is a hardware sale for ASUS and Intel.
- **The route:** the integrators who already sell inspection to these lines; the clause table is what they configure by hand today.
- **Why now:** a vision-language model that fits one workstation card is new, and Intel's own stack lists this one for it.
- **In the buyer's terms:** escape rate (defective boards that passed: 12 of 200 in Stage I), first-pass yield (good boards wrongly rejected: 16 of 201)[3], and the engineer's hours returned to the line.

Before[2]

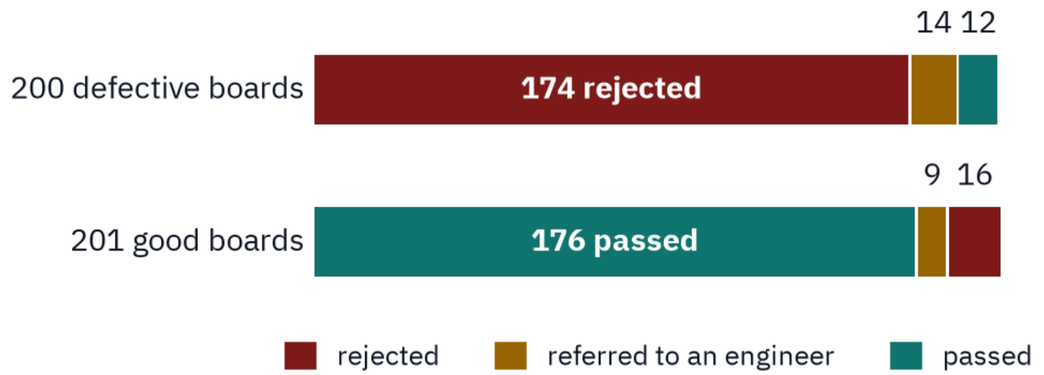
Read the customer's specification	Write the inspection rules by hand	Tune them on the line

"It takes us weeks": the persona's words, not a measurement.

After[1]

7.9 s parse the PDF into clauses	**12.2 min** train on good boards	The engineer reviews the clause table and the first batch: not timed

Measured on the laptop for one part of the demonstration specification.



What became of the Stage I test boards: the escapes and the false rejects a quality department counts.

[1] The 'after' times: results/stage1/bench.json (11 October 2026), on the laptop. [2] The 'before' is the persona's own words (page 3), not a figure. [3] Escaped and wrongly rejected boards: the same file. [4] No market-size or price figure is printed: none was found at a source.

FEASIBILITY

Risks, and why the plan survives them

The risks are known, named and planned for; the measured ones are printed with their numbers[1].

RISK	WHAT THE PLAN DOES
The machine's shape was not published.[2]	Retired: read on 6 November 2026, two whole cards in one bare-metal host, so the fallback for a share of a card was not needed.
Clause citation is weak: 34.1%, the target missed.	Met, and printed as measured (page 17). An unknown clause, or a class that is not the clause's, becomes "needs review", never a guess. FP8: no figure separates them. A good board's picture beside the crop is not yet built, and gets its own dated addition.
8.0% of good boards are rejected.	The judge clears few of the anomaly model's false alarms (2 of 28 flagged), and we say so. The thresholds stay on the test split, the only labelled data there is; the engineer's "not a defect" is a correction like any other.
A correction acts as a rule about the class (14 of 19 control boards moved, on the laptop).	Corrections are scoped to the part, capped at three, and printed on every card they touched. What a store of them does to the whole set, in both directions, is to be measured before 2 December 2026 at 11:30 PLAN
Measured extents are coarse.	The card prints what was measured and how; an area too small to outline is referred; the map stays at 256 px on the machine.
The answer contract has a known defect: 5 answers refused on the first asking in the final run.	The model may write no digit, and 4 padded to the token limit; 0 boards were referred for it. The designators comparison let the digit through: no figure separates them, and the final figure keeps the ban.
A specification written for public data looks staged.	It says so on its first page, in the console and here. The product is the clause schema and the citation, which a real specification fills the same way.
"How is this not Intel's defect explainer?"	Page 4: theirs explains a defect; ours binds each reject to the customer's own clause and limit, and keeps the engineer's override against it.
The machine's reservation ends 2 December 2026 at 11:30, before the video is due on 4 December 2026.	Scripted runs, results saved per stage and copied off as each ends, nothing that needs a camera. What is left of week 4 after that works from what was copied.

[1] Measured figures: results/stage2/final/FINAL.md; the corrections' figures, on the laptop: results/stage1/corrections.json (11 October 2026). [2] The machine: results/stage2/week1/machine.md.

References, and where to look

Every figure in this deck can be checked[2]. Code and documentation: https://github.com/BrianIsaac/spec-to-inspector · Demo video: https://youtu.be/1RTXhtMX2sg

1. ASUS UGen AI League Innovation Hackathon, rules and judging criteria. https://contest.bhuntr.com/en/39jg9vimiynrhlksze/home/
2. Intel Arc Pro B70 Graphics, specifications. https://www.intel.com/content/www/us/en/products/sku/245797/intel-arc-pro-b70-graphics/specifications.html
3. ASUS UGen Intel Arc Pro B70 32GB. https://www.asus.com/motherboards-components/ai-accelerator/ugen/ugen-arc-pro-b70-32g/
4. Anomalib 2.7.0, Apache-2.0. https://pypi.org/project/anomalib/2.7.0/
5. Rolih, Ameln, Vaidya and Akcay, "Divide and Conquer: High-Resolution Industrial Anomaly Detection via Memory Efficient Tiled Ensemble", 2024. https://arxiv.org/abs/2403.04932
6. VisA: Zou, Jeong, Pemula, Zhang and Dabeer, "SPot-the-Difference Self-Supervised Pre-training for Anomaly Detection and Segmentation", ECCV 2022. https://arxiv.org/abs/2207.14315 · data, CC BY 4.0: https://github.com/amazon-science/spot-diff
7. Qwen3-VL-8B-Instruct, Apache-2.0. https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct · FP8: https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct-FP8 · GGUF: https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct-GGUF
8. llama.cpp, MIT. https://github.com/ggml-org/llama.cpp
9. docling 2.135.0, MIT. https://pypi.org/project/docling/2.135.0/
10. PaddleOCR-VL-1.6, Apache-2.0. https://huggingface.co/PaddlePaddle/PaddleOCR-VL-1.6
11. Intel llm-scaler, supported models. https://github.com/intel/llm-scaler
12. vLLM recipes, pull request 1024, Intel Arc Pro B70 (XPU) configurations, merged 24 September 2026. https://github.com/vllm-project/recipes/pull/1024
13. Backend.AI, "Intel Arc meets Backend.AI", June 2026. https://www.backend.ai/blog/2026-06-intel-arc-pro-b70-meets-backend-ai
14. Intel Open Edge Platform 2026.2, release notes, 10 September 2026. https://docs.openedgeplatform.intel.com/dev/OEP-articles/release-notes.html
15. Intel Pro Day fact sheet, March 2026. https://download.intel.com/newsroom/2026/ClientComputing/intel-pro-day-fact-sheet.pdf
16. PCBA Standard-to-Real Challenge, ACM Multimedia 2026. https://2026.acmmm.org/site/grand-challenges.html · https://sites.google.com/cmlab.csie.ntu.edu.tw/asus-ntu-pcba-vqa-challenge/home
17. GLLS: "Global Logic and Local Search", 4 July 2026. https://arxiv.org/abs/2607.03817
18. VAND 4.0 challenge report: "Detecting Defects that Matter", 3 October 2026. https://arxiv.org/abs/2610.04392
19. TunnelMIND: "Training-Free Tunnel Defect Inspection and Engineering Interpretation", 30 April 2026. https://arxiv.org/abs/2604.27928
20. Aqrose, AIDI 3.4, 17 July 2026. https://www.aqrose.com/AIDI-3-4-is-Officially-Released-Intelligence-elevated-beyond-efficiency-id01372145.html · VISION 2026 notice, 3 September 2026. https://www.aqrose.com/Aqrose-to-Showcase-Industrial-AI-and-Vision-Language-Models-at-VISION-Stuttgart-2026-id05547135.html
21. Overview AI, OV Spark. https://overview.ai/products/ov-spark/ · Sparky's documentation. https://docs.overview.ai/docs/guides/ov-spark/sparky
22. ZEDEDA, "How to Build Flexible AI Quality Inspection for Manufacturing Using LLMs", 19 March 2026. https://zededa.com/blog/how-to-build-flexible-ai-quality-inspection-for-manufacturing-using-llms/
23. Intel, "How Many SR-IOV Virtual Functions Are Supported on IBCs Intel Arc Pro B70 and B50 Graphics?". https://www.intel.com/content/www/us/en/support/articles/000103382/graphics/intel-arc-pro-dedicated-graphics-family.html
24. Archived: https://github.com/intel/ipex-llm · https://github.com/intel/intel-extension-for-pytorch
25. In the repository: eval/PROTOCOL-stage2.md and its two additions; results/stage2/final/FINAL.md, comparison-fp8/, comparison-image-minimum/, comparison-designators/, week1/, week2/; docs/stage2.md.

The customer's own specification, cited on every reject.

[1] Each numbered address was fetched and answered on 11 October 2026 (`deck/build.sh --urls`). [2] Measured figures are in results/stage2/ and results/stage1/ of the repository, each written by a command of the repository's own.